

INSTITUTO TECNOLÓGICO SUPERIOR DEL OCCIDENTE DEL ESTADO DE HIDALGO

DIVISIÓN DE INGENIERÍA EN SISTEMAS COMPUTACIONALES

Manual Técnico de Firmware y Código Fuente Embebido

Pulsera Médica de Arteria Radial para Monitoreo Hemodinámico Continuo

Microcontrolador ESP32 Mini, Sensado Biofotónico MAX30102, Acelerometría MPU6050,
Servidor BLE GATT y Monitoreo Autónomo de Batería con Histéresis

Proyecto:

**Monitor Continuo No Invasivo de Signos Vitales (BPM, Presión Arterial PWA y SpO2) con
Cancelación Adaptativa de Ruido de Movimiento y Compensación Melánica Cutánea**

Asesores:

- Mtra. Lorena Mendoza Guzmán
- Mtra. Cristy Elizabeth Aguilar Ojeda

Alumno / Desarrollador:

- Alejandro Pérez

Grupo:

- 8.º Semestre - Grupo A
-

Contenido

- **1. Arquitectura de Firmware y Definiciones Globales**
 - 1.1 Librerías del Sistema, Mapeo de Puertos y Declaración de UUIDs GATT
 - 1.2 Búferes de Acumulación y Estructuras de Datos de Telemetría
 - **2. Subsistema de Sensores sobre Bus I2C Compartido (400 kHz)**
 - 2.1 Inicialización de Hardware e Interfaz I2C Rápida
 - 2.2 Parametrización Óptica y Registros Biofísicos del Sensor MAX30102
 - 2.3 Configuración de Registros y Adquisición Triaxial del Acelerómetro MPU6050
 - **3. Subsistema de Energía y Gestión de Batería**
 - 3.1 Conversión Analógica-Digital en ADC1 (GPIO 34) y Divisor Resistivo
 - 3.2 Máquina de Estados de Histéresis Clínica de Seguridad (3.40V / 3.50V)
 - **4. Subsistema de Comunicación Inalámbrica BLE GATT**
 - 4.1 Configuración del Servidor GATT y Servicio Diagnóstico
 - 4.2 Empaquetamiento y Notificación de la Trama Extendida de 20 Bytes (50 Hz)
 - 4.3 Característica de Telemetría de Batería y Banderas de Estado (UUID 0x2A19)
 - 4.4 Receptor Asíncrono de Comandos de Control (START / STOP)
 - 4.5 Callback de Reconexión Automática Reactiva ante Interrupción de Enlace
 - **5. Bucle Principal Concurrente y Temporización Sincrónica**
 - 5.1 Ciclo de Muestreo Biofotónico Estricto a 100 Hz y Monitoreo Energético Periódico
-

1. Arquitectura de Firmware y Definiciones Globales

1.1 Librerías del Sistema, Mapeo de Puertos y Declaración de UUIDs GATT

El firmware de la pulsera está implementado en C++ sobre el framework Arduino para el microcontrolador ESP32 Mini. Coordina de forma concurrente el muestreo del sensor óptico MAX30102, el acelerómetro MPU6050, la lectura de batería por ADC1 y la pila Bluetooth Low Energy (BLE).

Se definen los UUIDs estándar y personalizados del servicio médico y sus tres características operativas: telemetría extendida, control inalámbrico y estado de batería.

Código Fuente

```
#include <Wire.h>
#include "MAX30105.h"
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

// Definición de Identificadores Únicos Universales (UUIDs)
#define SERVICE_UUID          "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define DATA_CHAR_UUID       "beb5483e-36e1-4688-b7f5-ea07361b26a8" // Notificación 20 bytes
#define CONTROL_CHAR_UUID     "12345678-1234-1234-1234-123456789abc" // Comando START/STOP
#define BATTERY_CHAR_UUID     "00002a19-0000-1000-8000-00805f9b34fb" // Nivel de batería

// Direcciones I2C de hardware
#define MAX30102_ADDR         0x57
#define MPU6050_ADDR          0x68

// Registros de control MPU6050
#define MPU_PWR_MGMT_1         0x6B
#define MPU_ACCEL_CONFIG       0x1C
#define MPU_ACCEL_XOUT_H       0x3B

// Pines del sistema embebido
#define I2C_SDA_PIN            21
#define I2C_SCL_PIN            22
#define BATTERY_ADC_PIN        34 // Bloque ADC1 (inmune a contienda con radio BLE)

// Parámetros de conversión y seguridad de batería
#define ADC_REF_VOLTAGE        3.30f
#define ADC_RESOLUTION         4095.0f
```

```
#define V_DIVIDER_RATIO    2.0f    // Divisor simétrico 100k + 100k
#define V_BAT_CRITICAL     3.40f   // Umbral de suspensión preventiva
#define V_BAT_HYSTERESIS   3.50f   // Umbral de recuperación por recarga
```

1.2 Búferes de Acumulación y Estructuras de Datos de Telemetría

Para transmitir la tasa estricta de 100 Hz ópticos sin sobrecargar la pila de radio BLE, el firmware acumula pares de muestras consecutivas (2 muestras cada 20 ms). Cada muestra contiene los canales Rojo e Infrarrojo cuantizados a 18 bits.

Código Fuente

```
MAX30105 particleSensor;
BLEServer* pServer = NULL;
BLECharacteristic* pDataChar = NULL;
BLECharacteristic* pControlChar = NULL;
BLECharacteristic* pBatteryChar = NULL;

bool deviceConnected = false;
bool streamingActive = false;
uint16_t packetSequence = 0;

#define SAMPLE_PAIRS_PER_PACKET 2
uint32_t redBuffer[SAMPLE_PAIRS_PER_PACKET];
uint32_t irBuffer[SAMPLE_PAIRS_PER_PACKET];
uint8_t bufferIndex = 0;

int16_t accelX = 0;
int16_t accelY = 0;
int16_t accelZ = 16384; // 1.0g gravitatorio nominal en reposo

uint8_t batteryPct = 100;
uint16_t batteryMv = 4200;
bool lowBatteryActive = false;
unsigned long lastBatteryCheck = 0;
```

2. Subsistema de Sensores sobre Bus I2C Compartido (400 kHz)

2.1 Inicialización de Hardware e Interfaz I2C Rápida

Inicializa los puertos GPIO 21 (SDA) y GPIO 22 (SCL) configurando la velocidad de reloj a 400 kHz (Fast-Mode). Se establece enlace de sondeo para comprobar la presencia física de ambos circuitos integrados en sus direcciones serie correspondientes (`0x57` y `0x68`).

Código Fuente

```
void setupI2C() {  
    Serial.println("[I2C] Iniciando bus serie a 400 kHz...");  
    Wire.begin(I2C_SDA_PIN, I2C_SCL_PIN, 400000);  
  
    // Comprobación de presencia del sensor óptico  
    Wire.beginTransmission(MAX30102_ADDR);  
    if (Wire.endTransmission() != 0) {  
        Serial.println("[ERROR] MAX30102 no responde en dirección 0x57.");  
    } else {  
        Serial.println("[OK] MAX30102 detectado en bus I2C.");  
    }  
  
    // Comprobación del acelerómetro  
    Wire.beginTransmission(MPU6050_ADDR);  
    if (Wire.endTransmission() != 0) {  
        Serial.println("[ERROR] MPU6050 no responde en dirección 0x68.");  
    } else {  
        Serial.println("[OK] MPU6050 detectado en bus I2C.");  
    }  
}
```

2.2 Parametrización Óptica y Registros Biofísicos del Sensor MAX30102

Configura el circuito integrado MAX30102 para optimizar la captación fotométrica sobre la arteria radial. Se fija la corriente a 12.0 mA (`ledBrightness = 60`), el modo multi-LED simultáneo (Rojo e IR), la tasa de adquisición pura a 100 Hz (sin promediado FIFO) y el ancho de pulso a 411 microsegundos para resolución máxima de 18 bits.

Código Fuente

```
void configureMAX30102() {
    if (!particleSensor.begin(Wire, I2C_SPEED_FAST)) {
        Serial.println("[CRÍTICO] Fallo en particleSensor.begin(). Reintentando...");
        while (1) {
            delay(1000);
        }
    }

    byte ledBrightness = 60; // ~12.0 mA (corriente balanceada de penetración)
    byte sampleAverage = 1; // Sin promedio (1 muestra por conversión para alta resolución)
    byte ledMode = 2;       // Modo 2: Rojo (660nm) + Infrarrojo (880nm) simultáneos
    int sampleRate = 100;   // 100 Hz (resolución temporal = 10 ms)
    int pulseWidth = 411;   // 411 us de ancho de pulso (ADC de 18 bits)
    int adcRange = 4096;    // 4096 nA (rango dinámico óptimo fotométrico)

    particleSensor.setup(ledBrightness, sampleAverage, ledMode, sampleRate, pulseWidth, adcRange);

    particleSensor.setPulseAmplitudeRed(ledBrightness);
    particleSensor.setPulseAmplitudeIR(ledBrightness);
    particleSensor.clearFIFO();
    Serial.println("[OK] MAX30102 parametrizado a 100 Hz, 18 bits, corriente 12 mA.");
}
```

2.3 Configuración de Registros y Adquisición Triaxial del Acelerómetro MPU6050

Despierta el sensor inercial escribiendo `0x00` en el registro `PWR_MGMT_1` (`0x6B`) y ajusta el fondo de escala a $\pm 2g$ escribiendo `0x00` en `ACCEL_CONFIG` (`0x1C`). La función de lectura realiza una petición burst de 6 bytes para leer atómicamente A_x , A_y , A_z en complemento a dos con sensibilidad de 16,384 LSB/g\$.

Código Fuente

```
bool initMPU6050() {
    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(MPU_PWR_MGMT_1);
    Wire.write(0x00); // Despertar oscilador interno (salir de bajo consumo)
    if (Wire.endTransmission() != 0) return false;

    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(MPU_ACCEL_CONFIG);
    Wire.write(0x00); // Rango de escala completa a +/- 2g (16384 LSB/g)
    return (Wire.endTransmission() == 0);
}

void readMPU6050Accel() {
    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(MPU_ACCEL_XOUT_H);
    Wire.endTransmission(false); // Restart de bus sin liberar línea SDA
    Wire.requestFrom((uint8_t)MPU6050_ADDR, (size_t)6, true);

    if (Wire.available() >= 6) {
        accelX = (Wire.read() << 8) | Wire.read();
        accelY = (Wire.read() << 8) | Wire.read();
        accelZ = (Wire.read() << 8) | Wire.read();
    }
}
```

3. Subsistema de Energía y Gestión de Batería

3.1 Conversión Analógica-Digital en ADC1 (GPIO 34) y Divisor Resistivo

El monitoreo energético de la batería Li-Po de 3.7V utiliza un divisor de tensión simétrico formado por dos resistencias de precisión de $100\text{ k}\Omega$ conectadas al pin GPIO 34. Mediante un sobremuestreo digital de 8 muestras se calcula la tensión real de la celda de litio con resolución de 12 bits (0 a 4095).

Código Fuente

```
void checkBatteryStatus() {
    uint32_t adcSum = 0;
    for (int k = 0; k < 8; k++) {
        adcSum += analogRead(BATTERY_ADC_PIN);
        delayMicroseconds(50);
    }
    float rawAdc = (float)adcSum / 8.0f;
    float voltage = (rawAdc / ADC_RESOLUTION) * ADC_REF_VOLTAGE * V_DIVIDER_RATIO;
    batteryMv = (uint16_t)(voltage * 1000.0f);

    int pct = (int)(((voltage - V_BAT_CRITICAL) / (4.20f - V_BAT_CRITICAL)) * 100.0f);
    batteryPct = (uint8_t)constrain(pct, 0, 100);

    // Lógica de histéresis de seguridad clínica
    if (voltage < V_BAT_CRITICAL) {
        lowBatteryActive = true;
    } else if (voltage >= V_BAT_HYSTERESIS) {
        lowBatteryActive = false;
    }
}
```

3.2 Máquina de Estados de Histéresis Clínica de Seguridad (3.40V / 3.50V)

Evita oscilaciones en la señalización diagnóstica. Cuando la celda Li-Po cae por debajo de 3.40V, la caída de tensión altera la curva de corriente de los diodos LED del MAX30102. La bandera `lowBatteryActive` suspende la estimación de presión arterial en el cliente receptor, requiriendo que la tensión supere 3.50V de recarga por el puerto USB-C para reanudar el cálculo.

4. Subsistema de Comunicación Inalámbrica BLE GATT

4.1 Configuración del Servidor GATT y Servicio Diagnóstico

Inicializa el stack Bluetooth Low Energy bajo el nombre de red `Tensiometro_Pulsera`. Instancia el servicio GATT médico y registra los descriptores de cliente BLE2902 para habilitar las notificaciones asíncronas hacia la aplicación receptora.

Código Fuente

```
void initBLEServer() {
    BLEDevice::init("Tensiometro_Pulsera");
    pServer = BLEDevice::createServer();
    pServer->setCallbacks(new ServerCallbacks());

    BLEService *pService = pServer->createService(SERVICE_UUID);

    // Característica de datos: 20 bytes por notificación
    pDataChar = pService->createCharacteristic(
        DATA_CHAR_UUID,
        BLECharacteristic::PROPERTY_NOTIFY
    );
    pDataChar->addDescriptor(new BLE2902());

    // Característica de control (START / STOP)
    pControlChar = pService->createCharacteristic(
        CONTROL_UUID,
        BLECharacteristic::PROPERTY_WRITE
    );
    pControlChar->setCallbacks(new ControlCallbacks());

    // Característica de batería (READ y NOTIFY)
    pBatteryChar = pService->createCharacteristic(
        BATTERY_CHAR_UUID,
        BLECharacteristic::PROPERTY_READ | BLECharacteristic::PROPERTY_NOTIFY
    );
    pBatteryChar->addDescriptor(new BLE2902());

    pService->start();

    // Iniciar anuncios de radiofrecuencia
    BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
    pAdvertising->addServiceUUID(SERVICE_UUID);
```

```
pAdvertising->setScanResponse(true);  
pAdvertising->setMinPreferred(0x06);  
pAdvertising->setMinPreferred(0x12);  
BLEDevice::startAdvertising();  
  
Serial.println("[OK] Servidor BLE GATT iniciado bajo 'Tensiometro_Pulsera'.");  
}
```

4.2 Empaquetamiento y Notificación de la Trama Extendida de 20 Bytes (50 Hz)

Condensa sincrónicamente 2 muestras completas de luz roja e infrarroja (codificadas en enteros uint24 Big-Endian), el contador incremental de secuencia (uint16 Little-Endian) y las 3 aceleraciones inerciales del MPU6050 (enteros signados int16 Little-Endian). Esta trama fija de 20 bytes se emite a 50 Hz mediante el método `notify()`.

Código Fuente

```
void notifyTelemetry20Bytes(uint32_t r1, uint32_t ir1, uint32_t r2, uint32_t ir2) {
    if (!deviceConnected) return;

    readMPU6050Accel(); // Actualizar aceleraciones físicas sincrónicas

    uint8_t payload[20];

    // Muestra 1: Canal Rojo (uint24 Big-Endian)
    payload[0] = (r1 >> 16) & 0xFF;
    payload[1] = (r1 >> 8) & 0xFF;
    payload[2] = r1 & 0xFF;

    // Muestra 1: Canal Infrarrojo (uint24 Big-Endian)
    payload[3] = (ir1 >> 16) & 0xFF;
    payload[4] = (ir1 >> 8) & 0xFF;
    payload[5] = ir1 & 0xFF;

    // Muestra 2: Canal Rojo (uint24 Big-Endian)
    payload[6] = (r2 >> 16) & 0xFF;
    payload[7] = (r2 >> 8) & 0xFF;
    payload[8] = r2 & 0xFF;

    // Muestra 2: Canal Infrarrojo (uint24 Big-Endian)
    payload[9] = (ir2 >> 16) & 0xFF;
    payload[10] = (ir2 >> 8) & 0xFF;
    payload[11] = ir2 & 0xFF;

    // Contador de Secuencia (uint16 Little-Endian)
    payload[12] = packetSequence & 0xFF;
    payload[13] = (packetSequence >> 8) & 0xFF;

    // Aceleración Triaxial MPU6050 (int16 Little-Endian, 1g = 16384)
    payload[14] = accelX & 0xFF;
    payload[15] = (accelX >> 8) & 0xFF;
```

```
payload[16] = accelY & 0xFF;  
payload[17] = (accelY >> 8) & 0xFF;  
payload[18] = accelZ & 0xFF;  
payload[19] = (accelZ >> 8) & 0xFF;  
  
pDataChar->setValue(payload, 20);  
pDataChar->notify();  
packetSequence++;  
}
```

4.3 Característica de Telemetría de Batería y Banderas de Estado (UUID 0x2A19)

Empaqueta un payload de 4 bytes con el nivel porcentual (0–100%), la tensión en milivoltios (uint16 Little-Endian) y un byte de banderas de diagnóstico clínico donde el bit 1 indica alerta activa por batería baja.

Código Fuente

```
void notifyBatteryTelemetry() {
    if (!deviceConnected || pBatteryChar == NULL) return;

    uint8_t flags = 0;
    if (deviceConnected)    flags |= 0x01; // Bit 0: Enlace de radio activo
    if (lowBatteryActive)    flags |= 0x02; // Bit 1: Batería baja activa (<3.4V)

    uint8_t batPayload[4];
    batPayload[0] = batteryPct;
    batPayload[1] = batteryMv & 0xFF;
    batPayload[2] = (batteryMv >> 8) & 0xFF;
    batPayload[3] = flags;

    pBatteryChar->setValue(batPayload, 4);
    pBatteryChar->notify();
}
```

4.4 Receptor Asíncrono de Comandos de Control (START / STOP)

Manejador asíncrono que procesa las órdenes de inicio y parada recibidas desde la computadora. Ante **"START"**, purga el búfer FIFO del sensor MAX30102, resetea el contador de secuencia a cero y activa el streaming; ante **"STOP"**, suspende la transmisión para ahorrar energía.

Código Fuente

```
class ControlCallbacks: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
        String rxValue = pCharacteristic->getValue().c_str();
        if (rxValue.length() > 0) {
            if (rxValue == "START") {
                streamingActive = true;
                packetSequence = 0;
                bufferIndex = 0;
                particleSensor.clearFIFO();
                Serial.println("[BLE] Comando START procesado. Telemetría de 20 bytes iniciad
a.");
            } else if (rxValue == "STOP") {
                streamingActive = false;
                Serial.println("[BLE] Comando STOP procesado. Adquisición pausada.");
            }
        }
    }
};
```

4.5 Callback de Reconexión Automática Reactiva ante Interrupción de Enlace

En caso de pérdida de enlace inalámbrico por alcance de radiofrecuencia o cierre de la aplicación cliente, el callback `onDisconnect` reactiva inmediatamente los paquetes de anuncio (`startAdvertising`), permitiendo que el software vuelva a vincularse al instante sin necesidad de apagar físicamente la pulsera.

Código Fuente

```
class ServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
        deviceConnected = true;
        Serial.println("[BLE] Enlace establecido con cliente receptor.");
    }
    void onDisconnect(BLEServer* pServer) {
        deviceConnected = false;
        streamingActive = false;
        packetSequence = 0;
        bufferIndex = 0;
        Serial.println("[BLE] Enlace interrumpido. Reanudando transmisiones de anuncio...");
        pServer->startAdvertising(); // Reconexión reactiva inmediata
    }
};
```

5. Bucle Principal Concurrente y Temporización Sincrónica

5.1 Ciclo de Muestreo Biofotónico Estricto a 100 Hz y Monitoreo Energético Periódico

El bucle principal `loop()` se ejecuta en un esquema no bloqueante. Mediante temporización con `millis()` evalúa el nivel de batería cada 2 segundos, mientras vacía a máxima velocidad el búfer FIFO del MAX30102 para estructurar los pares de muestras de 100 Hz y emitir los paquetes de 20 bytes hacia el filtro adaptativo NLMS.

Código Fuente

```
void setup() {
  Serial.begin(115200);
  analogReadResolution(12); // ADC a 12 bits (0-4095)
  pinMode(BATTERY_ADC_PIN, INPUT);

  Serial.println("\n=====");
  Serial.println("  ESTACIÓN MÉDICA CARDIOVASCULAR - PULSERA ESP32 MINI");
  Serial.println("  Telemetría PPG (MAX30102) + Aceleración Triaxial (MPU6050)");
  Serial.println("=====");

  setupI2C();
  initMPU6050();
  configureMAX30102();
  initBLEServer();
  checkBatteryStatus();
}

void loop() {
  // 1. Monitoreo de batería periódico cada 2000 ms
  if (millis() - lastBatteryCheck >= 2000) {
    lastBatteryCheck = millis();
    checkBatteryStatus();
    notifyBatteryTelemetry();
  }

  // 2. Muestreo biofotónico e inercial síncrono a 100 Hz
  if (deviceConnected && streamingActive) {
    particleSensor.check();
  }
}
```

```
    while (particleSensor.available()) {  
        redBuffer[bufferIndex] = particleSensor.getRed();  
        irBuffer[bufferIndex] = particleSensor.getIR();  
        particleSensor.nextSample();  
        bufferIndex++;  
  
        if (bufferIndex >= SAMPLE_PAIRS_PER_PACKET) {  
            notifyTelemetry20Bytes(redBuffer[0], irBuffer[0], redBuffer[1], irBuffer[1]);  
            bufferIndex = 0;  
        }  
    }  
    delay(1);  
}
```